

Программирование и алгоритмизация

(МИСиС, осень-зима 2025)

Полевой Дмитрий Валерьевич
к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

Пространство имен (namespace)

- логически группирует объявления
- определяет область видимости
- может быть вложенным
- может быть анонимным
- открыто
 - допускается добавление имен в пространство в нескольких определениях

Создание пространства имен

```
namespace «имя_пространства_имен»  
{  
    // объявления и определения  
}
```

Создание пространства имен

```
namespace misis
{
    // Узнать число групп.
    int getGroupsCount();
}
```

Создание пространства имен

(пример вложенности)

```
namespace misis
{
    namespace job
    {
        int getGroupsCount();
    }
}
```

Добавление имен

(пример открытости)

```
// in parser.h  
namespace parser {  
    bool parse();  
}
```

```
// in lexparser.h  
namespace parser {  
    int getLexCnt();  
}
```

Доступ к именам в пространстве

доступ через оператор разрешения области
видимости ::

имя_пространства::имя

пример:

std::cout

std::filesystem::path

Глобальное пространство имен

- все, что не лежит в собственном пространстве имен
- доступ через :: (можно и нужно опускать)

using-директива (избегать)

вносит все имена из заданного пространства имен
в текущую область видимости

```
using namespace имя_пространства_имен;
```

пример:

```
using namespace std;  
cout << "Hello!" << endl;
```

using-объявление

(избегать)

ВВОДИТ имя в текущую область видимости

```
using  
    имя_пространства_имен::имя_члена;
```

пример:

```
using std::cout;  
  
cout << "Hello!" << std::endl;
```

using-псевдоним

объявляет псевдоним типа

- обычно, шаблонного
- м.б. шаблоном

```
using псевдоним_типа = тип;
```

```
template <class T>
```

```
using псевдоним_типа = тип;
```

using-псевдоним (примеры использования)

```
using Vec =  
    std::vector<int>;
```

```
Vec v;
```

```
template<class T>  
using Ptr = T*;
```

```
Ptr<int> x;
```

Соккрытие имен

```
int x(10); // глобальная переменная
```

```
void f() {
```

```
    int x(0); // скрывает глобальную
```

```
    x = 1; // присваивание локальной
```

```
    ::x = 5; // присваивание глобальной
```

```
}
```

Анонимное пространство имен

ограничение видимости для компоновщика

```
namespace  
{  
    // локальный для единицы трансляции код  
}
```

Рекомендации по использованию

- в рамках курса – избегать `using`
- избегать `using` в заголовочных файлах
- использовать `using`-псевдонимы
 - в файлах реализации (ограниченно и разумно)
 - по необходимости внутри функций/типов

ваши вопросы

Абстрактный тип данных (АТД)

- описывает набор операций
(логика использования)
 - создание/уничтожение
 - изменение
 - получение описания

примеры АТД:

число, строка, массив, стек, очередь

Структура данных

- способ организации и манипуляции данными (в памяти)
- реализация АД (в исходном коде)

примеры структур данных:

кортеж, массив, список, дерево, граф

Интерфейс

- спецификация услуг
 - семантический уровень (проектирование)
 - синтаксический уровень (исходный код)
- в узком смысле — набор всех функций
 - для типа
 - для модуля
 - для подсистемы

Инвариант типа

- **состояние экземпляра**
 - значения всех переменных
 - может изменяться методами
 - может изменяться прямым изменением полей
- определяется типом
(логика использования)

Инвариант типа

- **должен выполняться**
в начале и конце каждого метода
- может контролироваться
assert, специальные методы

АТД «число»

- создание, уничтожение, копирование
- сравнение ($=$, $\neq$, $<$, $\leq$, $>$, $\geq$)
- арифметические операции
- $-$, $+=$, $+$, $-=$, $-$, $*=$, $*$, $/=$, $/$
- ВВОД и ВЫВОД ($<<$, $>>$)

АТД «комплексное число»

$$z = \{re, im\} \quad \text{или} \quad z = re + i * im$$

$re \in \mathbb{R}$ - вещественная часть

$im \in \mathbb{R}$ - мнимая часть

$i^2 = -1$ - мнимая единица

инварианта нет

читать: Босс В. Лекции по математике: ТФКП (Т.09)

АТД «рациональное число»

$$r = \frac{num}{denum}, \text{ где}$$

$num \in \mathbb{Z}$ -целое и $denum \in \mathbb{N}$ - натуральное

$$\text{НОД}(num, denum) = 1$$

числитель и знаменатель не могут принимать
любые (целые) значения, т.е. существует инвариант

ваши вопросы

Матрица (в математике)

математический объект, записываемый в виде прямоугольной таблицы чисел и допускающий алгебраические операции (сложение, вычитание, умножение) между ним и другими подобными объектами. Обычно матрицы представляются двумерными (прямоугольными) таблицами.

а что такое таблица?

$f(0,0)$	$f(0,1)$	$f(0,2)$
$f(1,0)$	$f(1,1)$	$f(1,2)$
$f(2,0)$	$f(2,1)$	$f(2,2)$

Матрица (математически)

$$f: X \times Y \rightarrow C$$

где

$x \in X = [0, M - 1]$ - индекс строки

$y \in Y = [0, N - 1]$ - индекс столбца

отображение из «пространства индексов»
(полное декартово произведение)
в пространство «значений элементов»

АТД «матрица»

- создание, уничтожение, копирование
- получение доступа к элементу по индексу (iRow, iCol)
- получение размера по измерению
- изменение размера по измерению
- печать

Операции

- алгебраические (соответствующие размеры)
 - умножение матрицы на число
 - сложение двух матриц
 - умножение двух матриц
- поэлементные (одинаковые размеры)
 - сложение
 - умножение и деление

Дополнительные операции

- транспонирование
- срезы
- вставка/удаление столбца/строки
- перестановка двух строк/столбцов
- сложение двух строк/столбцов
- умножение на число строки/столбца

Приведенный индекс

- размеры матрицы
- массив элементов матрицы

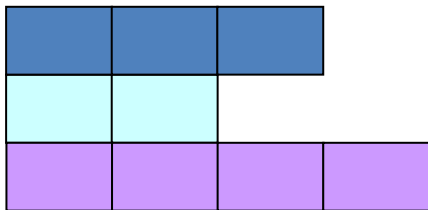
$$idx = i * N + j$$



почему
«массив массивов»
это не
«матрица»?

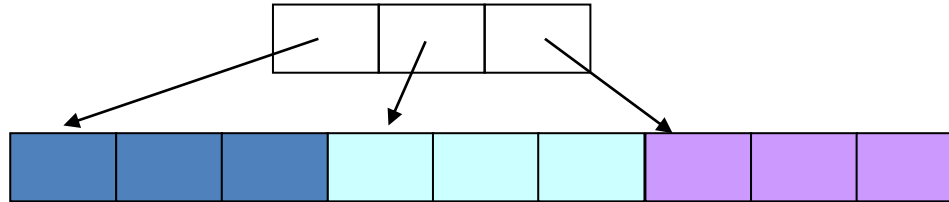
Массив массивов

- «ожерелье» (Jugged Array)
- элементы могут иметь разную длину



Массив строк/столбцов

- массив элементов матрицы
- массив индексов начала строк/столбцов



какое представление лучше?

Оценка реализации

- зависит от типового набора операций и распределения частот операций
- в массиве строк/столбцов «быстрая» перестановка строк/столбцов

ваши вопросы

Типы указателей

- обычный
- сингулярный (`nullptr`)
 - нельзя разыменовывать
 - можно проверять на равенство
- указатель, следующий за последним в массиве
 - нельзя разыменовывать
 - можно использовать в арифметике указателей и сравнениях

Перегрузка операторов доступа

`operator->()`

`operator*()`

позволяет использовать синтаксис указателей
для итераторов и «умных указателей»

Диапазон (объектов)

- $[first, last)$ состоит из всех объектов (элементов) от $*first$ до $*(last-1)$ включительно
- является допустимым
 - $last$ достижим из $first$
 - можно получить адрес каждого объекта
 - пустой диапазон $[p, p)$ является допустимым

Свойства диапазонов

- если непустой диапазон $[first, last)$ является допустимым, то $[first+1, last)$ является допустимым
- если $[first, mid)$ и $[mid, last)$ допустимые, то $[first, last)$ допустимые
- если $[first, last)$ допустимый и элемент mid достижим из $first$, $last$ достижим из mid , то $[first, mid)$ и $[mid, last)$ допустимые

Массив, как диапазон

- [beg, beg + SIZE)
- число элементов $SIZE = last - first$
- last, как признак конца обработки

пример:

```
for (p = first; p != last; ++p) {  
    if (value != *p) {  
        ...  
    }  
}
```

Регулярный тип

(ведет себя как встроенный)

- конструируемый по умолчанию
- копирование сохраняет равенство
- сравнение по значению (в т.ч. для частей)
- равенство и неравенство согласованы
- неравенство строгое и согласовано с равенством

Регулярный тип

(ведет себя как встроенный)

- `T(const T&) noexcept`
- `T(const T&&) noexcept`
- `operator=(const T&) noexcept`
- `operator=(T&&) noexcept`
- `operator==(const T&) noexcept`
- `operator<()` **и/или** `std::less<>`
- `hash<>`

Контейнер

- содержит свои элементы
- является регулярным типом
(если элементы регулярного типа)
- предоставляет доступ к элементам

Свойства элемента контейнера

- элемент принадлежит не более, чем одному контейнеру (семантика значений)
 - контейнеры не пересекаются
 - ограничение на объекты, а не на значения
- время жизни элемента не превышает время жизни контейнера
 - создается не раньше
 - уничтожается не позже

Иерархия концепций контейнеров

- контейнер произвольного доступа
- реверсивный контейнер
- однонаправленный контейнер
- контейнер (итератор ввода)

Итератор

- интерфейс между алгоритмами и структурами данных
(требования со стороны алгоритмов)
- регулярный тип (`operator=()` , `operator==()`)
- доступ к элементу контейнера
`operator*()` , `operator->()`
- навигация по контейнеру

Пример алгоритма поиска с итераторами

```
I find(I fst, I lst, const T& val) {  
    while (fst != lst && *fst != val) {  
        ++first;  
    }  
    return first;  
}
```

Пример использование поиска с итераторами

```
auto iV = find(b, e, v);  
if (e != iV) {  
    //  
}
```

ваши вопросы

Принцип идентичности

- соотношения равенства итераторов и равенства объектов

$$i1 == i2 \quad \leftrightarrow \quad \&*i1 == \&*i2$$
$$i1 == i2 \quad \rightarrow \quad *i1 == *i2$$

Константные и изменяемые итераторы

- константный итератор
 - доступ к константному объекту
 - константный объект (значительно реже)

пример:

```
const int* pData(0);
```

```
// !не константный итератор  
int* const pVariant(&pA);
```

Итератор произвольного доступа

- допускает написание алгоритма с произвольным доступом к контейнеру
- все манипуляции за амортизированное константное время

```
operator++(), operator--()  
operator+=(int)  
operator-()  
operator<()  
operator[] (int)
```

Двунаправленный итератор

- допускает написание многопроходного алгоритма

```
operator++()
```

```
operator--()
```


Однонаправленный итератор

- допускает написание однопроходного алгоритма

`operator++()`

- возможно существование более одного итератора для интервала

Итератор вывода

- однопроходное чтение (извлечение) элементов
 - не изменяет элементы
 - единственный итератор для диапазона
 - по диапазону нельзя пройти более одного раза

`operator++()`

Итератор ввода

- однопроходная запись элементов
 - единственный итератор для диапазона
 - по диапазону нельзя пройти более одного раза
 - не сравниваются (конец задан неявно)`operator++()`

пример:

```
*iV = x;
```

как соотносятся итераторы
и «обычные указатели»?

Пример алгоритма (copy)

```
OutI copy(InpI first, InpI last, OutI res) {  
    for (; first != last; ++res, ++first) {  
        *res = *first;  
    }  
    return res;  
}
```

Классы итераторов в STL

- `iterator` и `const_iterator`
- являются вложенными классами для классов контейнеров

пример:

```
vector<int>::iterator
```

```
list<int>::const_iterator
```

Вложенные типы

- определяются внутри определения типа (класс, структура)
- подчиняются спецификаторам доступа
- доступ с помощью оператора разрешения области видимости в т.ч. для `typedef`

Вложенные типы (пример)

```
class Stack {  
    public:  
        // ...  
    private:  
        struct Node {  
            // ...  
        };  
};
```


Вложенные типы (пример в STL)

```
std::vector<int>::iterator it(v.begin());  
auto itEnd(v.cend());  
auto it = std::begin(v);
```

Интервал элементов контейнера

`begin()` – первый элемент

`end()` – элемент, следующий за последним

`cbegin()`, `cend()` – `const` версии

пример:

```
vector<int> ms;
```

```
vector<int>::iterator i(ms.begin());
```

```
auto it = ms.cbegin();
```

Унификация интерфейса (функции)

`std::`

`begin() , end() , cbegin() ,
cend() , data() , size() , empty()`

в качестве параметров

- контейнеры
- указатели (встроенные массивы)
- `std::initializer_list`

Пример поиска

```
auto it = std::find(std::cbegin(dt),  
    std::cend(dt), k);  
if (std::cend(dt) != it) {  
    // найден элемент со значением k  
    ...  
}
```

ваши вопросы

Структура (struct)

- составной пользовательский тип
- определяет набор (почти) произвольных типов
- именованный доступ

Пример структуры (плохой)

```
struct Vec2i {  
    int x;  
    int y;  
};
```

`Vec2i v1; // ! не инициализирован`

`Vec2i v2 = {3, 5};`

Умолчательный инициализатор поля

- указывается в определении типа с помощью
= или { }

пример:

```
struct Vec2i {  
    int x = 0;  
    int y = 0;  
};
```


Доступ к членам структур

- . – оператор доступа к члену
- -> – оператор разыменования члена структуры (для указателя на экземпляр структуры)

пример:

```
pt.x = 1;
```

```
ppt->x = ppt->y;
```

```
std::pair<T1, T2>
```

```
template<typename T1, typename T2>
```

```
struct pair {
```

```
    T1 first;
```

```
    T2 second;
```

```
};
```

```
auto p = std::make_pair("text", 5);
```

Структуры – POD

(Plain Old Data)

- копирование экземпляров «по памяти»
является допустимым

пример:

```
auto pt_next = pt_prev;  
pt_curr = pt_next;
```

ваши вопросы

Строгое перечисление (`enum class`)

- пользовательский тип
 - строгая проверка типов компилятором
 - отсутствие неявных преобразований
- набор именованных константных целочисленных значений
- возможно уточнение типа хранения

Строгое перечисление (пример)

```
enum class KeyWord {  
    kAsmM,  
    kAuto,  
    kBreak  
};
```

```
KeyWord w(KeyWord::rAsm);
```

```
void f(const KeyWord& currWord);
```

Строгое перечисление (пример)

```
enum class Status : std::uint32_t {  
    kUndefined = -1,  
    kOff = 0,  
    kOn = 1,  
    kDisable = 255  
};
```

Перечисление (enum)

избегаем использования

- приводится к целому в выражениях
- имена экспортируются в окружающую область видимости
- может не иметь имени

Инициализация

= // C инициализация

() // C++98 инициализация

{ } // C++11, универсальная инициализация

пример:

```
int a = 15; // C-стиль
```

Стиль инициализаторов

{ } для агрегатов и «по необходимости»
= для одиночных значений

пример:

```
int a = 15;
```

пример:

```
struct N {  
    N* p = nullptr;  
};
```

Неявные преобразования

- **повышение**

`bool → int`

`enum → int или long`

- **обеспечение точности**

`int + long → long`

`int + double → double`

- **усечение (округление)**

`int n(f);`

`int k{f}; // ошибка компиляции`

Примеры округления

- до целого (с усечением)

```
int size(d) ;
```

- до ближайшего

```
int size(d + 0.5) ;
```

- нетривиально для отрицательных

Явные преобразования

`static_cast<Тип> (выражение)`

преобразование родственных типов при
компиляции

пример:

```
int roundV(static_cast<int>(double_v));
```

Явные преобразования

`const_cast<Тип> (выражение)`

снятие `const` и `volatile`

пример:

```
char* p_beg(const_cast<char*>(p + n));
```

Явные преобразования

`reinterpret_cast<Тип> (выражение)`

интерпретация объекта

пример:

```
char* p(reinterpret_cast<char*>(adr));
```

Преобразования в стиле C

избегать

(Тип) выражение

Тип (выражение)

пример:

```
int roundVal(int(double_val + 0.5));
```


ваши вопросы

Инвариант типа

- **состояние экземпляра**
 - значения всех переменных
 - может изменяться методами
 - может изменяться прямым изменением полей
- определяется типом
(логика использования)

Инвариант типа

- **должен выполняться**
в начале и конце каждого метода
- может контролироваться
assert, специальные методы

Как поддерживать
инвариант типа?

Управление доступом

`public`

открытые члены, имена доступны везде

`protected`

защищенные члены, имена доступны только в
собственных функциях и функциях-членах
производных классов

`private`

закрытые члены, имена доступны только в
собственных функциях-членах

Определение класса

```
class ClassName {  
    public:  
        // открытые методы  
    private:  
        // закрытые методы  
private:  
    // закрытые данные  
};
```

Пример (точка)

```
class Point {  
    public:  
        // методы  
    private:  
        int x_{0}; //< x-координата  
        int y_{0}; //< y-координата  
};
```

Доступ к членам классов

- аналогично структурам с учетом
- . – оператор доступа к члену
- -> – оператор разыменования члена структуры (для указателя на структуру)

Конструктор

- имя метода совпадает с именем типа
- не возвращает значений
- вызывается автоматически
- создает экземпляр (инициализирует)

пример:

```
Point::Point(const int x, const int y);
```

Список инициализации

- сконструированный объект д.б. инициализирован
- умолчательный инициализатор игнорируется

пример:

```
Point::Point(const int x, const int y)
    : x_(x)
    , y_(y)
{
}
```

Порядок инициализации

- определяется порядком объявления переменных в классе
- не зависит от порядка в списке инициализации
- инициализируйте переменные в порядке объявления

Перегрузка функций

- использование одного имени для операций с разными наборами аргументов
- одна из форм полиморфизма

пример:

```
int min(const int lhs, const int rhs);
```

```
double min(const double lhs,  
           const double rhs);
```

Перегрузка конструктора (пример)

```
class Point {  
    public:  
        Point(const int x, const int y);  
        Point(const Point& point);  
        // ...  
};
```

Конвертирующий конструктор

- имеет ровно один параметр
- МОЖЕТ вызываться неявно
! используйте `explicit` для запрета

пример:

```
explicit T(const int value)
```

Деструктор

- имя метода строится из имени типа
- не принимает аргументов
- не возвращает значений
- разрушает экземпляр (при необходимости)
- вызывается автоматически

Деструктор (пример)

```
// point.h
class Point {
    public:
        // ...
        ~Point() ;
        // ...
};
```

```
// point.cpp
Point::~~Point() {
    // ...
}
```


Деструктор

- всегда объявляем
- явно указываем использование сгенерированного варианта (default)

пример:

```
~Complex() = default;
```

Указатель на экземпляр `this`

- доступен внутри экземплярных методов
- содержит адрес текущего экземпляра

пример:

```
T& operator=(const T& rhs) {  
    //...  
    return *this;  
}
```

Умолчательные версии методов (пример)

```
class Complex {  
    public:  
        Complex() = default;  
        Complex(const Complex&) = default;  
        ~Complex() = default;  
        // ...  
};
```

ваши вопросы

Операторная функция

имя начинается с ключевого слова `operator`,
после которого идет сам оператор

пример:

```
operator+
```

```
operator++
```

объявление соответствует грамматике языка
(количество и состав аргументов)

Перегрузка операторов

внутренняя (метод)

`T operator@(const T& rhs)`

внешняя (свободная функция)

`T operator@(const T& lhs, const T& rhs)`

Унарный оператор

- выражение @aa или aa@
- нестатический метод без аргументов
aa.operator@()
- свободная функция с одним аргументом
operator@(aa)

Префиксная и постфиксная формы унарных операторов

префиксный оператор @aa
возвращает результат вычисления

постфиксный оператор aa@
возвращает исходное значение аргумента

Префиксные и постфиксные операторы (сигнатуры)

- префиксный оператор

`T& operator++()`

`T& operator--()`

- постфиксный оператор

`T operator++(int)`

`T operator--(int)`

Префиксный унарный оператор (пример)

```
T& T::operator++() {  
    // содержательная работа  
    return *this;  
}
```

Постфиксный унарный оператор (пример)

```
T T::operator++(int) {  
    T initValue(*this)  
    ++(*this);  
    return initValue;  
}
```

Бинарный оператор (aa @ bb)

метод с одним аргументом

`aa.operator@ (bb)`

свободная функция с двумя аргументами

`operator@ (aa, bb)`

Ограничения перегрузки операторов

- нельзя создать «новый» оператор
- нельзя изменить число аргументов
- нельзя изменить приоритет

Предопределенный смысл

- должен сохраняться
- должен быть переопределен для всех вариантов операторов

пример:

`++a` означает `a += 1` означает `a = a + 1`

Сохранение смысла

- для автономных версий операторов $a@b$ всегда предоставляйте присваивающую версию $@=$
- для операторов, допускающих перестановку аргументов, предоставляйте все соответствующие версии

Принцип минимальности интерфейса

- минимизируйте число функций, непосредственно манипулирующих представлением объекта
- реализуйте в классе только те операторы, которые модифицируют значение первого аргумента

Минимальность интерфейса (пример)

- **в классе**

```
T& operator+=(const T& rhs)
```

- **вне класса**

```
T operator+(const T& lhs, const T& rhs) {  
    T sum(lhs);  
    sum += rhs;  
    return sum;  
}
```

Перегрузка операторов (рекомендации)

- перегружайте умеренно и осмысленно
- используйте операторы для имитации привычной формы записи
- минимизируйте минимизируйте создание временных объектов

Перегрузка операторов (рекомендации)

- обеспечивайте полный набор перегруженных операторов
- минимизируйте количество независимых операторов

пример:

```
operator==
```

```
operator!= // реализовать через operator==
```

Смешанная арифметика

```
RatN val(0); // рациональное число  
2 + val;  
val + 2;
```

требуется наличия операторов

```
RatN operator+(const RatN&, const int);  
RatN operator+(const int, const RatN&);
```

ваши вопросы

Классы ввода/вывода

- `ostream`
 - поток вывода (класс)
 - глобальные объекты `cout`, `cerr`, `clog`
- `istream`
 - поток ввода (класс)
 - глобальный объект `cin`

Проблема перегрузки операций ВВОДА И ВЫВОДА

- левым аргументом является экземпляр класса стандартной библиотеки
- модификация класса левого аргумента невозможна или нежелательна

Перегрузка ввода/вывода

- **ВЫВОД**

`ostream& operator<<(ostream&, T)`

или

`ostream& operator<<(ostream&, const T&)`

- **ВВОД**

`istream& operator>>(istream&, T&)`

Оператор вывода (пример)

```
bool T::writeTxt(ostream& ostr) const;

ostream&
operator<<(ostream& ostr, const T& rhs) {
    rhs.writeTxt(ostr);
    return ostr;
}
```

Друзья

- метод
 - имеет доступ к закрытой части объявления
 - находится в области видимости класса
 - должен вызываться для экземпляров класса (имеется указатель `this`)
- друг (`friend`-функция или `friend`-класс)
 - имеет доступ к закрытой части объявления

friend-функция

- указывается в объявлении класса, другом которого она является
- может указываться в любой секции (`public`, `protected`, `private`)

Практика использования `friend`

использовать `friend` не рекомендуется

допустимо

- реализация сильно связанных концепций
- удовлетворение обоснованных требований по оптимизации

Friend-функция (пример)

```
class Matr;  
  
class Vect {  
    friend Vect operator*(const Matr&, const Vect&);  
};  
  
class Matr {  
    friend Vect operator*(const Matr&, const Vect&);  
};  
  
Vect operator*(const Matr& lhs, const Vect& rhs) {  
    //...  
}
```

ваши вопросы

Константность (`const`)

- замена препроцессора
- строгий контроль типов
- защита данных
- оптимизация

Константность и защита данных

- инициализация в процессе выполнения
- изменение не предусмотрено
- компилятор пресекает попытки потенциального изменения (проверка присваиваний и модификации)

Константные переменные

- не изменяются после определения

пример:

```
const ptrdiff_t size(vec.size());
```

Константные параметры

- не изменяются внутри функции

пример:

```
int min(const int lhs, const int rhs)
ostream& write(ostream& ostrm, const T& val)
```

Константные поля

- не изменяются после создания объекта
- инициализируются
 - в списке инициализации
 - умолчательным инициализатором

Константные поля (пример)

```
class FixedArr {  
    public:  
        FixedArr(const int size);  
    private:  
        const int size_{0};  
        ///...  
};  
  
FixedArr::FixedArr(const int size)  
    : size_(size) {  
    ///...
```

Константные методы

- сохраняют логическую константность
 - не изменяют состояния полей кроме mutable
- могут вызываться для константных экземпляров
- отличаются от неконстантных методов с тем же набором параметров

Константные методы (пример)

```
class FixedArr {  
    public:  
        //...  
        int size() const;  
        //...  
};  
  
int FixedArr::size() const {  
    //...
```

Поля-ссылки

- сильно связанные данные (классы)
- инициализируются в списке инициализации
- блокируют создание умолчательного конструктора

Поля-ссылки (пример)

```
class Point {  
    public:  
        explicit Point(const Axes& axes);  
    private:  
        const Axes& axes_;  
        ///...  
};  
  
explicit Point::Point(const Axes& axes)  
    : axes_(axes)  
    ///...
```


ваши вопросы

Чем отличается
копирование
от
присваивания?

Отличие копирования от присваивания

```
T::T(const T&obj)
```

```
// состояния ещё нет
```

```
T& operator=(const T&obj)
```

```
// состояния уже есть
```

```
// при "проблемах" хочется сохранить
```

Исключения и operator=

```
T& T::operator=(const T& obj) {  
    // возможны проблемы  
    // как "откатить изменения"?  
    return *this;  
}
```

Помним про $x=x$ (присваивание самому себе)

```
T& T::operator=(const T& obj) {  
    if (this != &obj) {  
        // ВОЗМОЖНЫ ПРОБЛЕМЫ  
        // как "откатить изменения"?  
    }  
    return *this;  
}
```

Cxema copy-and-swap

```
T& T::operator=(const T& obj) {  
    if (this != &obj) {  
        swap(*this, T(obj))  
    }  
    return *this;  
}
```

Схема copy-and-swap (не оптимально)

```
T& T::operator=(T obj) {  
    swap(*this, obj)  
    return *this;  
}
```

ваши вопросы